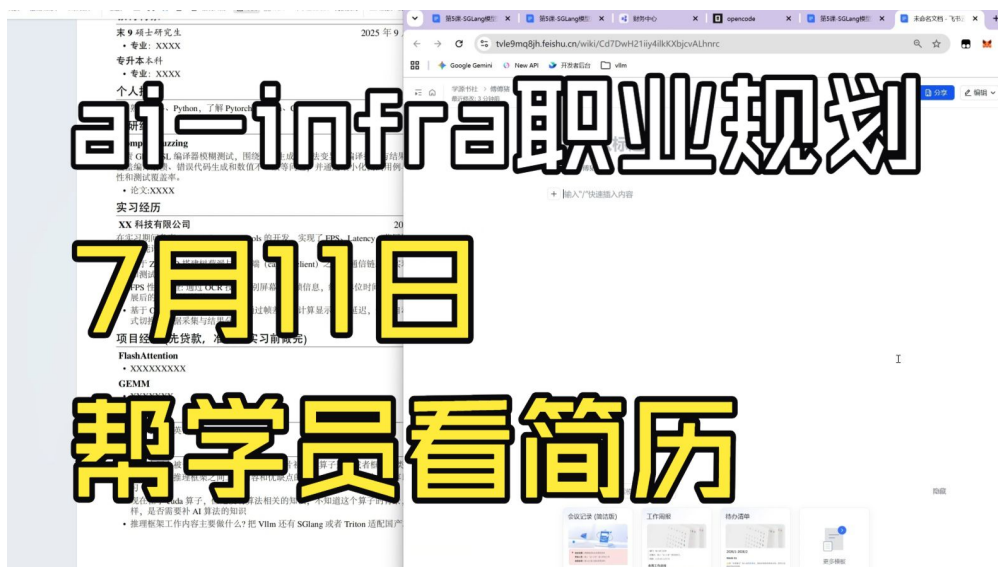


视频笔记

AI-Infra 规划课和模拟面：帮学员看简历

五道口纳什 & Codex

2026 年 7 月 18 日



视频作者/频道：我是傅傅猪

发布日期：2026-07-11

视频时长：21 分 39 秒

视频链接：<https://www.bilibili.com/video/BV1nBNw6XE47>

目录

1 开场与简历概览	2
1.1 本章小结	2
2 学历与职业方向讨论	3
2.1 学历担忧与老师回应	3
2.2 芯片初创 vs 中场算子开发	3
2.3 本章小结	3
3 Compiler Fuzzing 项目深挖	3
3.1 项目目标与测试流程	3
3.2 多卡支持与个人贡献	4
3.3 论文有效性的关键追问	4
3.4 本章小结	5
4 推理框架工作内容	5
4.1 vLLM/SGLang 适配国产芯片的插件机制	5
4.2 推理框架的四大工作类型	5
4.3 本章小结	6
5 AI 基础与算子开发	6
5.1 AI 基础薄弱的提醒	6
5.2 算子开发的工作内容与优化套路	6
5.3 面试现场写算子的要求	6
5.4 本章小结	7
6 简历问题与改进建议	7
6.1 实习经验的定位问题	7
6.2 科研经历的”贷款”问题	7
6.3 老师的整体建议	8
6.4 本章小结	8
7 总结与延伸	8
7.1 讲者结尾讨论	8
7.2 蒸馏的核心主张	8
7.3 实践含义与交叉链接	9
7.4 开放问题	9
7.5 拓展阅读	9

1 开场与简历概览

视频是 AI-Infra 规划课的一对一模拟面试，老师在屏幕共享状态下逐段审阅学员简历并给出口头点评。开场先确认学员现状：

- 当前在一家 GPU 公司实习，做性能工具开发（实习时间 2025 年 10 月–2026 年 2 月）。
- 学校里在跟导师做论文，方向是 compiler fuzzing（编译器模糊测试）。
- 论文方法迁移自导师之前的 Katech、ONNX Runtime 等工作，目标是放到 GPU DSL 编译器上。
- 实习公司的主营业务其实是存储主控芯片，GPU 性能工具只是一个延伸小业务（USB 3.0 转接 HDMI/DP 设备的 FPS、延迟、花屏检测），与普遍意义上的”GPU 性能”概念不同。

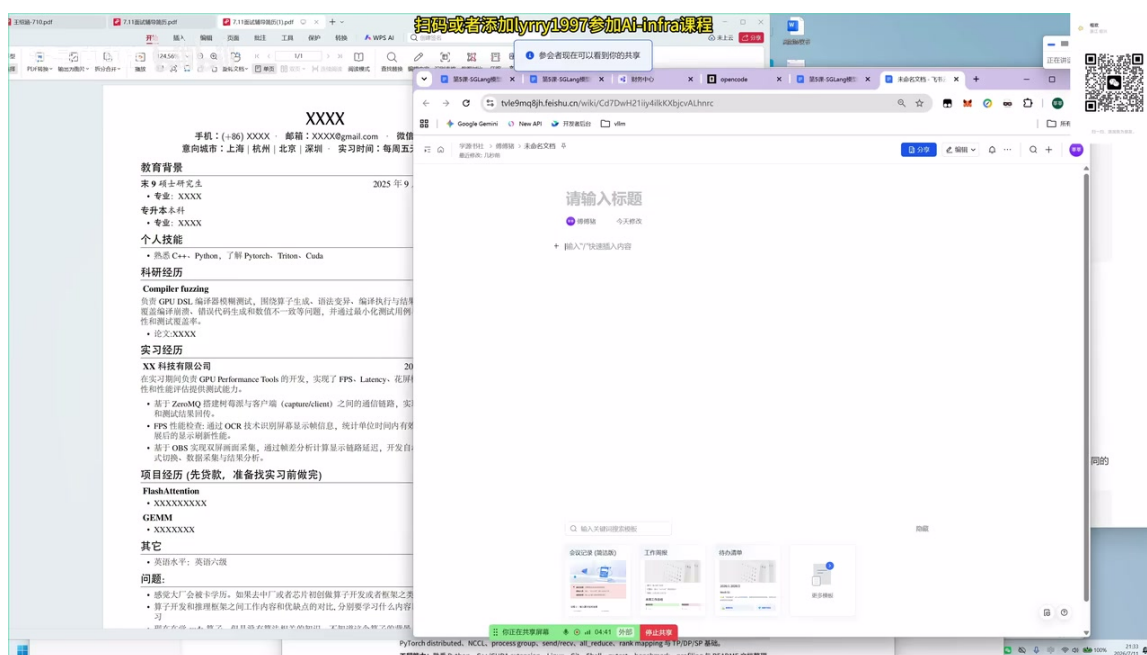


图 1: 开场屏幕共享的简历完整概览，包含教育背景、个人技能、科研经历与实习经历四块。¹

简历内容快照

简历主体四块：教育背景（专升本本科 → 985 硕士在读，2025 年 9 月–2028 年 7 月）；个人技能（C++、Python、PyTorch、Triton、CUDA）；科研经历（Compiler fuzzing，2026 年 3 月至今）；实习经历（XX 科技有限公司，GPU Performance Tools 开发，含 ZeroMQ 通信链路、FPS/Latency/花屏检测）。

1.1 本章小结

开场建立了一个典型的”学员带着不完整简历来问方向”的咨询场景。老师的提问策略是先确认业务上下文（GPU 公司是做存储主控的，性能工具是周边业务），避免按”通用 GPU 性能”的预设去问，这是面试官快速校准问题框架的常用技巧。

¹ 视频画面时间区间：00:00:00–00:02:00。

2 学历与职业方向讨论

2.1 学历担忧与老师回应

学员明确表达”感觉大厂会卡学历”，因为本科是专升本的，硕士才考上 985。老师的回应很直接：

学历问题的处理原则

学历已经没办法改变，担心没有意义。唯一能做的是把自己的技术打磨得更精。面试官真正在意的是”你能不能干活”，而不是学历标签本身。

2.2 芯片初创 vs 中场算子开发

学员问”如果去做算子开发或者芯片初创做算子开发”该怎么选。老师区分了两类公司的不同要求：

- **芯片初创公司**：现在市面上有几十上百家，质量参差不齐。关键判断标准是**有没有实际可以稳定出货的产品**。如果去的公司只有模拟器、没有流片产品，在模拟器上编程”学到最后其实也没学到什么东西”。
- **中场（成熟大厂）**：例如商汤、Momenta 这类已经有完整产品线的公司，算子开发岗位更稳定，但竞争也更激烈。

芯片初创公司的隐藏风险

没有实际产品的芯片初创公司，会让工程师长期在模拟器上开发。模拟器和真实硬件之间永远有差距，等真正流片回来发现问题，可能已经错过了学习窗口。投递前务必确认该公司是否有可稳定出货的芯片产品。

2.3 本章小结

学历讨论的核心不是”如何消除学历差距”，而是”如何在学历不可逆的前提下最大化技术信号”。芯片初创 vs 中场的取舍，本质上是用”公司产品成熟度”作为筛选门槛，把”看起来在做芯片”和”真的有芯片”区分开。

3 Compiler Fuzzing 项目深挖

老师对简历上的 Compiler fuzzing 项目做了较深入的提问，这是面试中”项目深挖”环节的典型范例。

3.1 项目目标与测试流程

项目核心目标是 GPU DSL 编译器的模糊测试，找编译器的 bug。完整测试流程如下：

1. **算子等价改写**：对一个算子做语义等价的变形（例如矩阵转置两次，语义不变）。
2. **约束构造**：用符号求解器（symbolic solver）描述算子输入变量之间的约束关系。

- 3. **参数生成**：基于约束，符号求解器生成大量满足约束的参数。
- 4. **执行比对**：把同一组参数喂给原始算子和等价变形后的算子，对比结果。
- 5. **缺陷上报**：若语义相同但结果不同，说明编译器在改写过程中存在优化错误或 bug，上报。

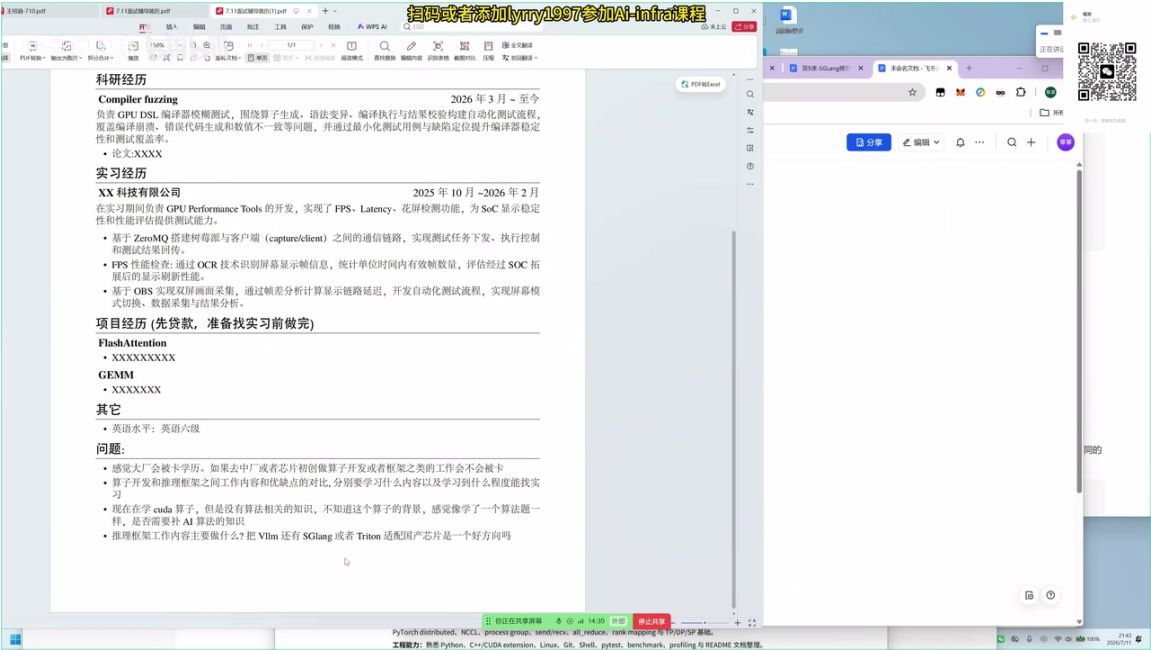


图 2：屏幕共享中简历的科研经历与实习经历两段，老师在追问测试流程的实现细节。²

3.2 多卡支持与个人贡献

学员提到该工具支持 NV、AMD、升腾、摩尔线程等多家厂商的卡。老师追问”你负责的工作是什么”，学员确认整个流程是自己独立做的，简历上的文字描述是用 GPT 生成的”凑数 PPT”，实际项目还没写完一半。

简历描述的诚信红线

学员主动承认”这段文字介绍是我用 GPT 随便生成的，还在开始阶段”。老师明确指出”应聘的时候不能瞎写”。即便项目还在进行中，简历描述也必须如实反映已完成的部分，否则面试官深挖时会立刻暴露。

3.3 论文有效性的关键追问

老师抛出一个对论文方向至关重要的问题：**如果投递之前还没测出任何编译器 bug，如何证明方法的有效性？**学员回答”找不出来就投差点（会议），找出来就投好的”，这是导师的建议。

老师进一步追问：**如果找到了 bug，应该由谁修复？**学员回答”做软工的不负责修复，爆出来经厂商确认后，在论文里报告 bug 数量、确认数、修复数”。老师指出这其实更像**测试岗位**的工作模式：只负责发现问题，不负责解决问题。

²视频画面时间区间：00:04:00–00:09:00。

测试岗 vs 开发岗的工作模式差异

测试岗：拿着其他方式写出来的算子，跟自己实现的算子做精度匹配，不匹配就报给开发。不需要自己构造语义输入和约束参数。**开发岗**：负责算子优化、改错、精度对齐。学员的 compiler fuzzing 项目偏向测试岗模式——构造输入找问题，但不解决问题。这对面试官来说“项目没动手”的信号比较强。

3.4 本章小结

这一节展示了面试官深挖项目的标准套路：目标 → 流程 → 个人贡献 → 关键挑战 → 成果证明。学员在“个人贡献”和“成果证明”两个环节都暴露了弱点（GPT 凑数 + 还没找到 bug）。老师把项目归类为“测试岗模式”，是给学员定位问题的核心信号。

4 推理框架工作内容

学员提到对“推理框架主要工作 vs 极浪 train 适配芯片”方向感兴趣，老师详细拆解了推理框架工程师的工作内容。

4.1 vLLM/SGLang 适配国产芯片的插件机制

老师先解释了 vLLM 的插件机制：芯片厂商在生产芯片后写一堆算子，然后以插件形式给 vLLM 添加后端，一般不会改 vLLM 的主流程（显存块管理、请求调度、量化等）。具体做法是用 patch 方法把 vLLM 申请显存的接口替换成自家芯片的对应接口。

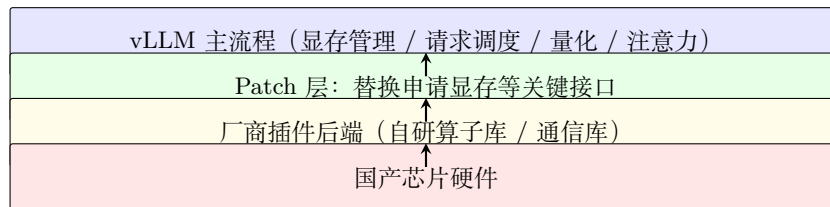


图 3: vLLM 适配国产芯片的插件机制：主流程不动，通过 patch 接口接入厂商后端。

4.2 推理框架的四大工作类型

老师把推理框架工程师的工作归纳为四大类（适配国产芯片作为第五类单独列出）：

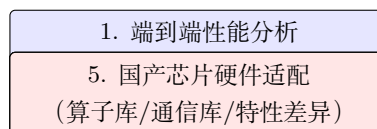


图 4: 推理框架工程师的五大工作类型。

端到端性能分析是最大头的工作：业务端反馈某模型在特定场景下慢或吞吐不够，工程师要复现场景、定位热点（计算慢、通信慢还是访存慢），再做针对性修改。

版本升级适配处理 vLLM 升级时的接口变更、功能删除或迁移带来的兼容性问题。

模型接入针对云厂商或有自研大模型的业务：模型结构与 vLLM 默认不匹配时，要让 vLLM/SGLang 能识别新结构并正确推理。

私有功能开发满足公司特定业务需求但开源社区没有的功能，通常不提交到上游。

推理框架工作的核心心智模型

推理框架工程师不是“写框架的人”，而是“让框架在自家业务和硬件上跑得好的人”。最大头的工作是性能分析（找瓶颈）+ 适配（接口/模型/硬件），而不是从零造轮子。

4.3 本章小结

推理框架工作内容的拆解对学员来说是定位自己方向的关键参照系：如果做“适配国产芯片”，主要工作是 patch 接口 + 算子库适配；如果做“通用推理框架”，最大头是端到端性能分析。这两类工作的技能栈和面试要求不同。

5 AI 基础与算子开发

5.1 AI 基础薄弱的提醒

学员提到“现在在学 CUDA，但没有算法相关知识、AI 算法的知识”。老师追问“AI 算法是哪一类”，学员回答“大概懂一些名词，看过 transformer 论文，但代码层面没太接触过”。

大模型加速岗位的 AI 基础红线

做大模型加速却不知道大模型是什么，是明显的硬伤。即使做的是 infra/算子方向，也必须熟悉 transformer 结构、注意力机制、KV cache 等基础概念，否则无法理解为什么某些算子需要特殊优化。

老师明确建议：transformer 这类基础必须补起来。

5.2 算子开发的工作内容与优化套路

学员正在学 CUDA，刚学到矩阵乘法。老师让学员描述对算子开发的理解，学员的回答其实已经命中核心：要了解底层硬件结构，利用访存特性（L1/L2）加速。

算子开发的核心优化套路

算子优化的“套路”无非就那几种：

- 能用 L1 访存就用 L1，放不下就用 L2。
- L2 还放不下就做切分（tiling），把数据分块。
- 利用硬件特性（访存模式、bank conflict 规避、向量化读写）加速。

熟悉之后，大部分算子优化都是这些套路的组合。

5.3 面试现场写算子的要求

老师明确给出算子开发岗位的面试门槛：

- 简历写了 matmul 或 transpose，面试时一定让你当场写一个。

- 标准不是”达到最高性能”，而是”能写出来 + 把优化策略讲清楚”。
- 时间充裕时会让你现场实现，时间紧就只问优化策略。

算子开发面试的”能写出来”门槛

”能写出来”指的是在屏幕共享状态下，从空文件开始实现一个 matmul 或 transpose，并能讲清 L1/L2 访存、tiling、向量化等优化点。这是面试官判断”是否真的写过 CUDA 代码”的最直接方式。课程学习能让你达到这个门槛，但高手是实战练出来的。

5.4 本章小结

AI 基础是 infra 岗位的隐性门槛，算子开发的核心是”访存层级 + 切分”的优化套路。面试现场写算子是硬性要求，简历上写了什么就会被要求实现什么。

6 简历问题与改进建议

6.1 实习经历的定位问题

老师对学员实习经历的核心评价是”不太相关”——做的是发现问题（FPS/Latency 检测），不是解决问题（算子优化、精度对齐）。这导致实习经历在算子开发岗位面试中信号弱。

6.2 科研经历的”贷款”问题

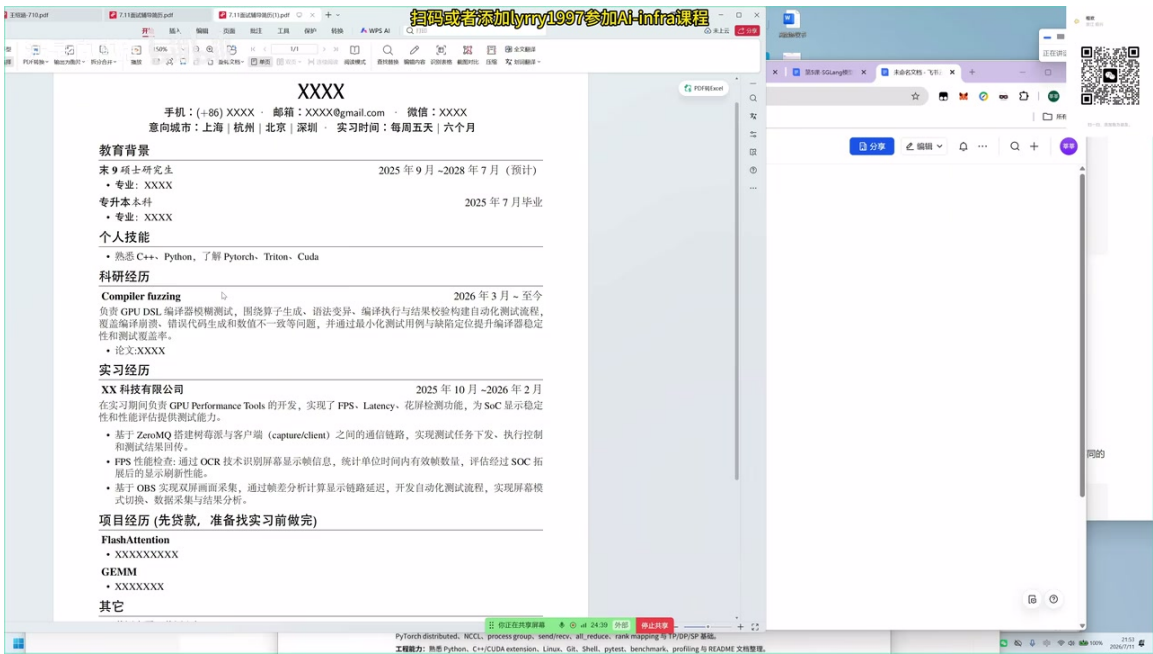


图 5: 简历顶部完整内容，老师在最后总结时再次回到这份简历做整体点评。³

老师指出科研经历的两个核心问题：

³视频画面时间区间：00:18:00-00:21:00。

科研经历的两大隐患

问题一：没动手。只负责发现问题（找 bug），不负责解决问题（修 bug/算子优化/精度对齐），整个流程偏向测试岗，缺少开发含量。

问题二：还没产出。商用编译器（NV/AMD/升腾/摩尔线程）都还没真正测出 bug，简历上写的成果是“贷款出来”的——预期未来会有，但现在没有。

6.3 老师的整体建议

- 整份简历都是“贷款出来”的，需要抓紧把实际产出做出来。
- 学员当前定位是“明年才找实习”（12 月或明年），还有时间补。
- 老师给学员“定定位问题”：科研经历像测试岗、AI 基础薄弱、算子开发还在学。
- 课程学完能达到“找实习”的水平，但要达到“高手”水平必须靠实战。

“高手是实战练出来的”

老师原话：“没有一个算子高手是课程教出来的，都是实战当中去锻炼出来的。”课程的作用是达到“能找实习”的门槛，之后的成长依赖真实业务问题的反复打磨。

6.4 本章小结

简历问题的核心是“信号弱”：实习像测试、科研是贷款、AI 基础薄弱。改进路径是**把贷款变成现货**——真正测出 bug、真正修过算子、真正写过 transformer 相关代码。

7 总结与延伸

7.1 讲者结尾讨论

老师在视频结尾给学员的总结性建议包含三层：

1. **定位问题**：科研经历像测试岗（只发现不解决），AI 基础薄弱，算子开发还在学。
2. **时间窗口**：明年才找实习，还有时间把“贷款”变成“现货”。
3. **课程 vs 实战**：课程能让你达到“找实习”门槛，但算子高手必须靠实战练出来。

7.2 蒸馏的核心主张

本视频的核心教学信号

简历信号强度 = 实际产出 + 能讲清楚 - 贷款成分。

简历上写的每一项都会被面试官深挖：写了 matmul 就会被要求现场写；写了 compiler fuzzing 就会被问测试流程、个人贡献、成果证明。如果某项还是“贷款”（预期未来会有但现在没有），深挖时会立刻暴露。

7.3 实践含义与交叉链接

对学员：本视频展示的面试深挖套路（目标 → 流程 → 贡献 → 挑战 → 成果）适用于任何技术岗位的简历项目。准备简历时按这个套路自检每一项。

对面试官：快速校准业务上下文（“这家 GPU 公司其实是做存储主控的”）是问对问题的前提。对芯片初创公司的筛选，“有没有稳定出货产品”比“技术栈是否匹配”更重要。

交叉链接：算子开发的 L1/L2 访存与切分优化，与 vLLM 推理框架的“端到端性能分析”在心智模型上同源——都是先定位瓶颈（计算/通信/访存），再决定优化策略。推理框架工程师虽然不写算子，但要理解算子优化套路才能正确定位热点。

7.4 开放问题

- 学员的 compiler fuzzing 方法在“没找到 bug”时如何证明有效性？这是论文写作的硬问题，本视频没给解决方案。
- 国产芯片适配的“算子库差异”具体指什么？视频没展开，需要补充具体案例。
- “算子高手靠实战”中的“实战”具体指哪些场景？是生产环境的性能调优，还是竞赛/开源贡献？视频没明确。

7.5 拓展阅读

- vLLM 官方文档关于插件机制与后端适配的说明：vLLM Documentation。
- Compiler fuzzing 经典工作可参考 Katech、ONNX Runtime 的相关论文，了解方法迁移路径。
- CUDA 算子优化的 L1/L2/tiling 套路，可参考 NVIDIA CUDA C++ Programming Guide 的 Memory Optimization 章节。